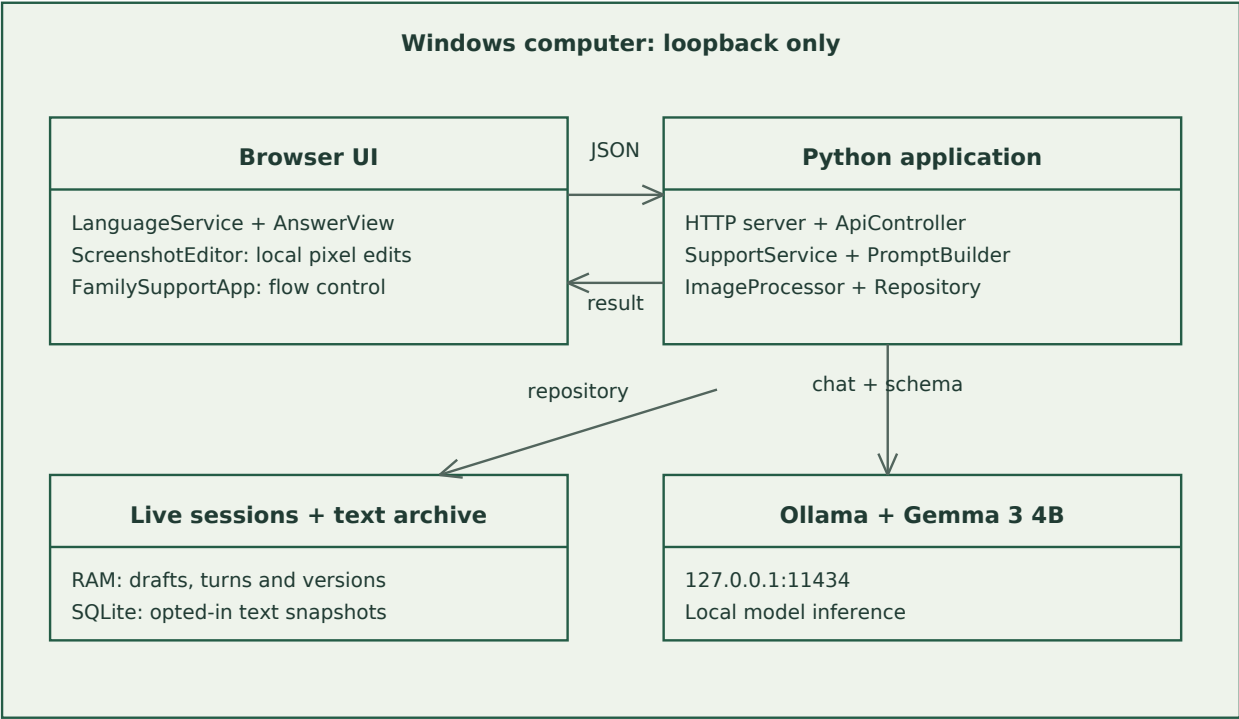


Family Tech Support Assistant

Technical learning guide · Version 1.2.0 · October 2026

Architecture, class-based design and all 14 UML diagram types

A practical guide to a Windows application that helps a family member understand an everyday computer or phone problem, confirm the assistant’s interpretation, and follow one small step at a time.



Built with Python, browser JavaScript, SQLite, Pillow, Ollama and Gemma 3 4B. Interface languages: English, Hindi and Gujarati. The implementation favors small classes and explicit state over extra frameworks.

Scope: educational local prototype. Automated workflow checks use a deterministic model double. They do not establish real Gemma accuracy, language fluency or speed on the target PC. The UML profile and timing views are explicitly illustrative.

Reading route and contents

Read pages 3–8 first to understand the system. Pages 9–22 introduce the fourteen UML diagram types one by one. Pages 23–32 explain implementation, verification, troubleshooting and extensions. Pages 33–34 add the SQLite schema and the supported phone workflow. The guide includes concrete invariants and exercises so you can connect a diagram to running code.

Pages	Topic
3–4	Problem, requirements and technology choices
5–8	Architecture, responsibilities, user flow and design patterns
9–15	Structural UML: class, object, component, composite structure, package, deployment and profile
16–22	Behavioral UML: use case, activity, state machine, sequence, communication, interaction overview and timing
23–25	API contract, session versions and prompt design
26–28	Screenshot pixels, multilingual UX and privacy boundaries
29–32	Tests, Windows setup, learning exercises, glossary and sources
33–34	SQLite schema/lifecycle and using a phone with the Windows helper

A useful way to study

Start with one user action, such as confirming a corrected understanding. Trace it from a button event into an HTTP endpoint, then into SupportService, the model adapter and finally the repository. Next read the class, sequence and state diagrams together: they answer who owns the behavior, in what order it runs, and which states can change.

Notation and honesty

Solid boxes do not automatically mean processes. A class is a code abstraction; a component is a logical responsibility; a deployment node is a running environment. The package diagram groups modules conceptually: the actual Python code remains in the single app package. These differences help prevent an educational diagram from being mistaken for an implemented distributed system.

1. Problem and product requirements

The person using the application

The intended user may be unfamiliar with settings, error messages and technical language. They need a readable interface, short instructions, control over what is shared, and an easy way to say that the assistant misunderstood or that a step failed. The developer needs code that is easy to inspect and change without building a large platform.

Requirement	Implemented behavior
Language	Choose English, Hindi or Gujarati before creating a conversation; interface labels and requested answer language match.
Screenshots	Crop, cover selected pixels in black, zoom the view and undo up to three edits before committing a PNG.
Understanding	Analyze first; show problem, evidence and uncertainty. User edits fields, then confirms before the first action.
Failed steps	Persist failed outcome and a note before requesting another action. Reject exact normalized repeats of failed instructions.
One step	Answer schema distinguishes step, question and escalation; prompting requests one reversible action.
Temporary data	Opted-in text snapshots persist in SQLite until deleted. Live RAM sessions have a 30-minute TTL. Screenshots are excluded from the database.

Quality goals

Favor understandable code, bounded memory, clear failure states and predictable feedback. Model correctness remains uncertain even when a response has valid JSON. A generated instruction is advice for the user to assess; the app does not execute device commands.

Acceptance example

A Gujarati-speaking user attaches a Wi-Fi screenshot, covers a visible phone number, corrects “router is broken” to “only this laptop is affected,” confirms the review, and later records “the menu did not contain that option.” That correction and outcome become explicit context for the next reply.

2. Technology and trade-offs

Technology	Why it is used
Python 3.11+	Dataclasses, HTTP transport, JSON, locks and a straightforward Windows entry point.
Pillow >=10.4,<13	Validate image bytes, orient images, resize to 1,600 pixels maximum edge and rebuild a metadata-free JPEG.
HTML / CSS / JavaScript	Small browser interface with focused classes, native forms and a native modal dialog; no Node runtime needed by users.
Ollama + Gemma 3 4B	Local multimodal inference through /api/chat; a JSON schema requests either understanding or an answer.
ReportLab (docs only)	Optional tool to regenerate this PDF with editable vector drawings. It is not a runtime dependency.
Playwright + axe (QA only)	Browser workflow and accessibility checks. Synthetic responses isolate application behavior from model quality.

Hardware-aware choices

The user’s target is Windows with 16 GB RAM and an RTX 3050 with 4 GB VRAM. Context is set to 4,096 tokens, output to 384 tokens, and only one inference request is accepted at a time. These choices limit work but do not prove a model fits entirely in VRAM. Ollama may place work in system memory; image understanding can be slow. CPU-only mode is available.

Other reasonable options

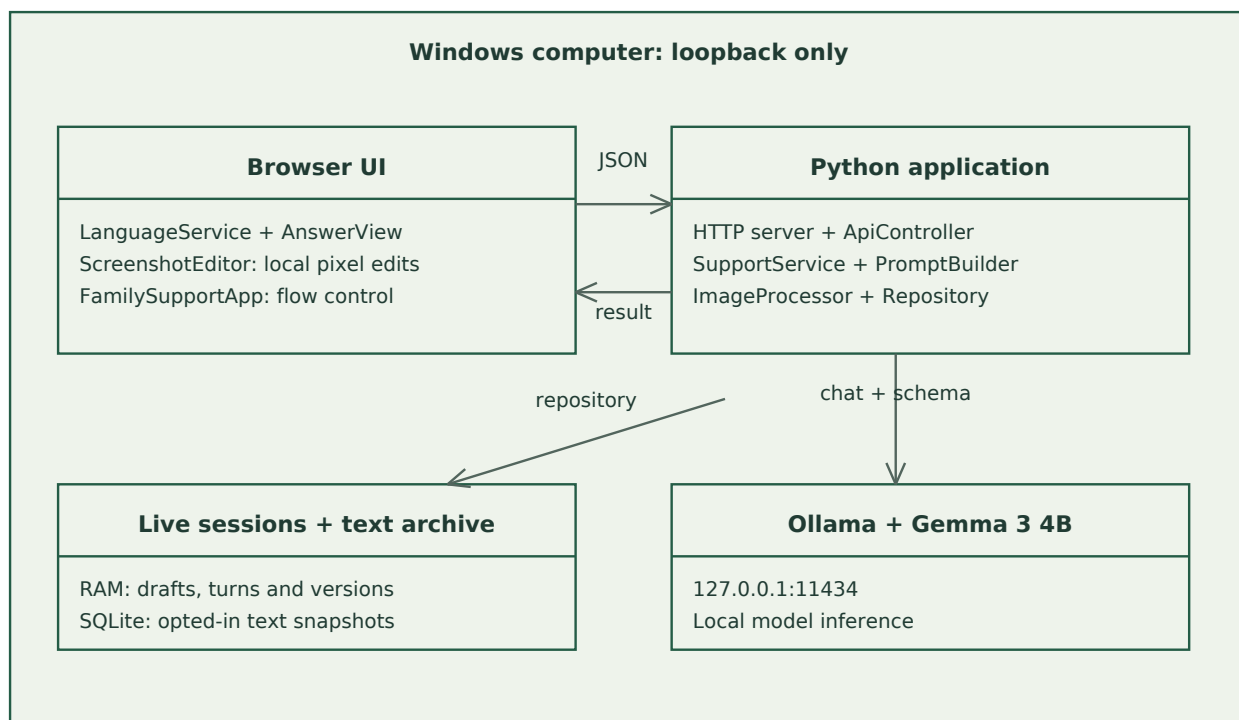
FastAPI can add typed HTTP contracts when endpoints become more complex. React or Vue can help when interface state becomes much larger. SQLite now supplies local text history without a new runtime package. A desktop wrapper can hide the browser/server setup, but brings packaging and update work. Each should solve an observed problem before it becomes a dependency.

Why a monolith is enough

One person, one PC and a small number of temporary conversations do not justify queues, a service mesh or multiple deployed backends. Logical boundaries still matter inside one process: the transport does not decide prompts, and the model adapter does not own conversation state.

3. System architecture

The diagram separates browser memory, Python memory and model inference. All application network destinations are loopback addresses on the same PC.



Read the arrows

The browser sends JSON to the Python API. SupportService validates the requested state, prepares the prompt and calls OllamaClient. OllamaClient sends an analysis or answer schema to the local Ollama endpoint. The service validates the returned fields and commits only after a successful response and a matching session version.

Important boundaries

Original screenshot bytes stay in browser memory during editing. Only the committed PNG enters an application request. Python rebuilds it as a JPEG; an initial draft retains that sanitized image until confirmation succeeds. Later screenshots are supplied for that turn only. Ollama may independently retain request context or logs; the application cannot guarantee operating-system-wide secure erasure.

Failure containment

A model timeout releases the inference lock and leaves the current persisted draft or turns intact. A feedback request commits separately, so failure of the following model request does not erase the report. Reset during inference removes the session; the later append fails instead of recreating it.

4. Responsibilities and source map

File / class	Responsibility
app/main.py · Application	Composition root: construct settings, collaborators, controller and loopback server.
app/server.py · RequestHandler	Bound request bodies; check Host and Origin; serve exact static allowlist; map errors to JSON.
app/controller.py · ApiController	Map routes to service methods.
app/service.py · SupportService	Analyze, correct, confirm, ask, feedback, solve and reset use cases; inference lock.
app/models.py	Immutable Conversation, Turn, Understanding and SupportAnswer values; output validation.
app/repository.py · SessionRepository	Live snapshots; versions; capacity/TTL; optional SQLite text archive on successful mutations.
app/prompts.py · PromptBuilder	Language, approved understanding, failed-action summaries and bounded complete exchanges.
app/images.py · ImageProcessor	Validate, orient, downscale and sanitize incoming pixels.
app/ollama.py · OllamaClient	Local HTTP adapter, schemas and useful error mapping.
app/static/app.js	ApiClient, AnswerView and FamilySupportApp.
app/static/editor.js	ScreenshotEditor owns pixel edits and committed image bytes.
app/static/translations.js	LanguageService and complete static en/hi/gu catalogs.
app/history.py · HistoryRepository	Local SQLite text snapshots, search, read and deletion.
app/static/history.js · HistoryView	Saved-history dialog; read-only archived chats.

A simple dependency rule

Put decisions about the support workflow in SupportService, storage behavior in SessionRepository, and presentation in browser classes. Avoid a controller that directly opens images, builds prompts and edits sessions. That would make each change harder to test and explain.

5. User flow and UX principles

1 — Describe

Choose a language and affected device. Enter a short description. Optionally open the screenshot editor. The file does not upload automatically; use the edited screenshot only after checking private details. The initial action asks for an understanding, not a solution.

2 — Check understanding

Gemma returns a problem statement, evidence and uncertainty. Three editable fields are visible. The user can replace an incorrect fact before confirmation. When fields differ, the UI saves the correction first and then requests the first answer. A failed confirmation preserves the corrected draft for retry.

3 — Try one step

The answer separates the action, reason and check. The user can send an observation, ask for an explanation, record a failed result, or explicitly mark the problem solved. History labels distinguish pending, failed, unclear and worked. No text classification guesses whether an action failed.

Principle	Concrete application
Visibility of state	Three-stage progress, busy text, reply count, result badges and connection status.
User control	Editable understanding, optional image, crop/redact/undo, explicit reset and solve.
Error prevention	Limits, disabled duplicate controls, confirmation before action, failed-action context.
Recognition	Visible field labels and history; instructions separate from explanatory text.
Accessibility	Semantic forms, visible focus, native modal, coordinate inputs, status/alert roles and responsive layout.

Claims to avoid

The design follows selected usability and accessibility principles. A passing automated scan is not full WCAG certification. Language quality, screen reader behavior and usefulness for the intended family member require direct testing.

6. Design patterns without overengineering

Layered architecture

Transport, application behavior, domain values and infrastructure adapters are separate responsibilities inside one application. Layers explain ownership. They are not separate servers, and no framework is required to maintain the boundaries.

Repository

SessionRepository is the single place that reads and replaces conversations. Its lock protects compound state operations. A service receives an immutable snapshot and later commits against its version. HistoryRepository separately saves opted-in text snapshots to SQLite. Keeping live sessions and archives separate avoids trying to resume drafts whose screenshots were deliberately not stored.

Adapter and protocol

OllamaClient translates the application's model operations into local HTTP requests. ModelClient describes the interface. Tests inject FakeModel; production injects OllamaClient. This isolates transport failures and avoids calling a real model just to test version logic.

Constructor injection

Application constructs the collaborators and passes them into SupportService. The service does not secretly create a network client during a unit test. Explicit dependencies are easy for a junior engineer to trace; a dependency-injection container would add little here.

Immutable values and composition

Frozen dataclasses represent snapshots. Repository operations use dataclasses.replace rather than mutating a shared Turn. Browser and Python classes combine collaborators; inheritance is limited to HTTP framework integration and the protocol relationship.

Optimistic concurrency

A request carries a version. A successful mutation increments it. If a request was prepared from an older snapshot, the repository refuses the commit. This is a concurrency technique, not a guarantee that requests or feedback are idempotent.

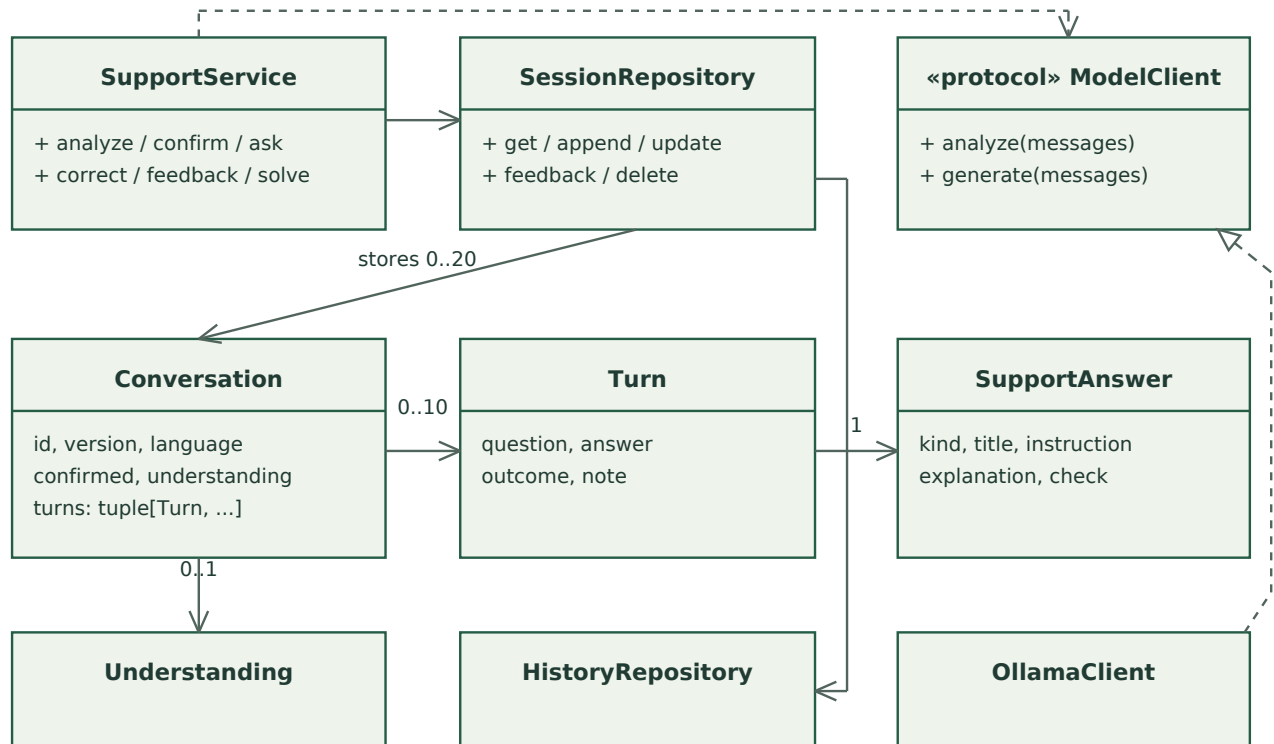
Keep abstractions earned

The project deliberately avoids a generic plugin framework, event bus or abstract base class for every value. Introduce a new abstraction when there is a second implementation or a clear testing boundary, not simply because a design-pattern name exists.

7. Class diagram

UML learning view · Release 1.2.0 · drawings are simplified for readability

A class diagram shows static types, their responsibilities and relationships. It is the best first UML view when studying class-based code.



How to read this diagram

Boxes show types and selected members. Arrows show references or use; multiplicities show bounds. The hollow triangle marks OllamaClient realizing ModelClient. Conversation has up to ten Turns, each referencing one SupportAnswer.

Where it appears in the code

Domain boxes are in `models.py`; use cases in `service.py`; snapshots in `repository.py`; model types in `ollama.py`. ImageProcessor and PromptBuilder appear in the composite structure view.

Design lesson

Keep data validation close to its value type, but put multi-object workflow decisions in a service. A domain object should not make an HTTP request just to construct itself.

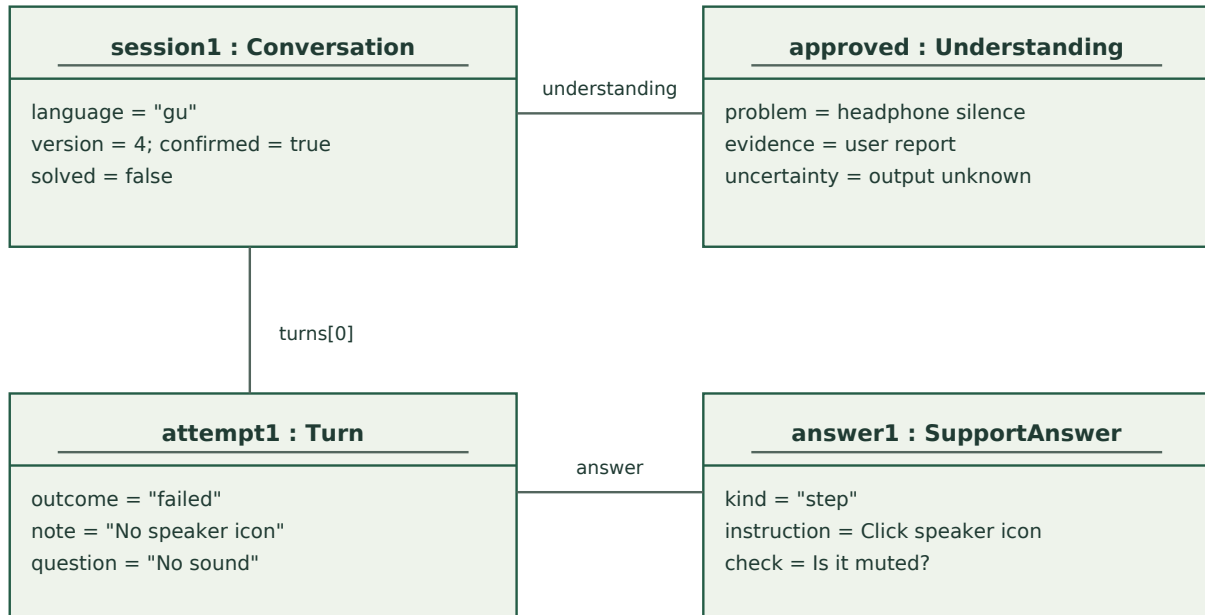
Try it yourself

Add a hypothetical preferred text size to Settings. Explain which classes need that value and which should not depend on it.

8. Object diagram

UML learning view · Release 1.2.0 · drawings are simplified for readability

An object diagram is a snapshot of specific instances, not a list of all possible classes.



How to read this diagram

Underlined names use instance : Class notation. Lines link values at one moment. This example shows a Gujarati session after analysis (version 1), correction (version 2), confirmation (version 3) and feedback (version 4). The failed action and note remain in the first Turn.

Where it appears in the code

SessionRepository.sessions maps a random session ID to a Conversation snapshot. Conversation.understanding holds the approved fields. Conversation.turns stores immutable Turns, each with a SupportAnswer. The sample text is illustrative data.

Design lesson

Class diagrams describe what can exist. Object diagrams show what actually exists in a particular state. They are helpful when checking whether an error should change the version, a draft or a previous outcome.

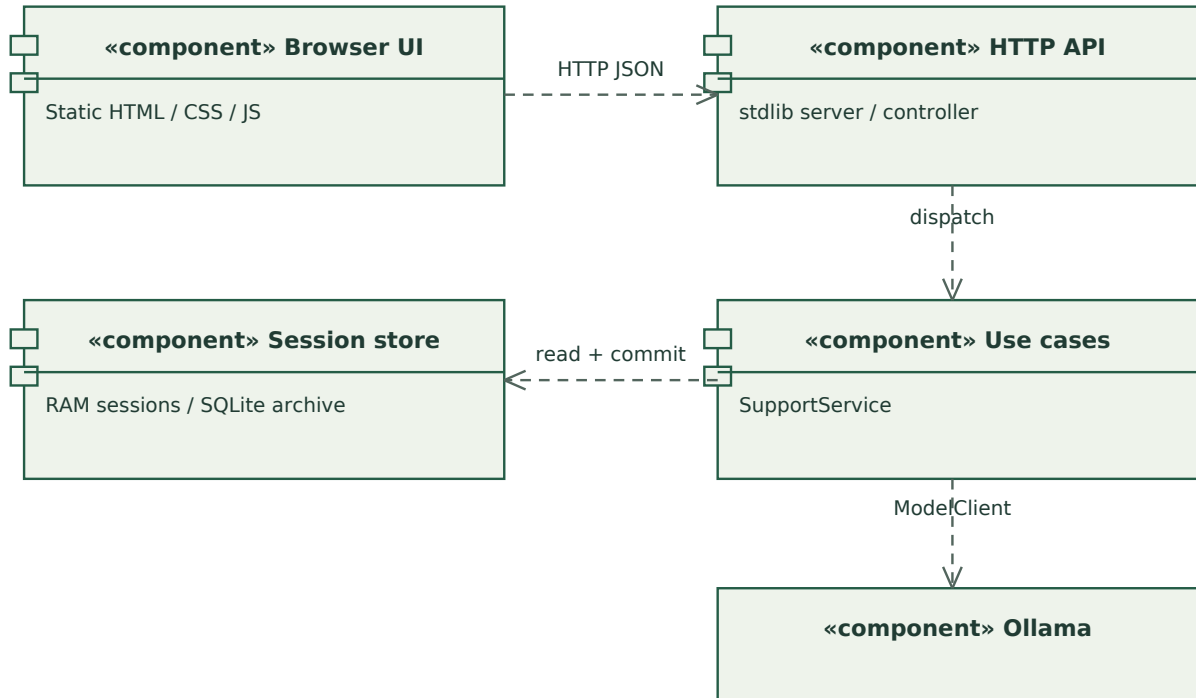
Try it yourself

Draw the snapshot immediately after a confirmation timeout. Which fields remain present, and why should confirmed still be false?

9. Component diagram

UML learning view · Release 1.2.0 · drawings are simplified for readability

A component diagram groups logical software responsibilities and their dependencies.



How to read this diagram

Tabbed boxes are components. Dashed dependencies indicate what a component needs. Browser UI uses the HTTP API; the API dispatches use cases; use cases depend on a session store and the model boundary. Logical components can all run within the same Python process.

Where it appears in the code

Browser files form the UI component. RequestHandler and ApiController form the HTTP boundary. SupportService coordinates use cases. SessionRepository supplies live memory storage and delegates opted-in saves to HistoryRepository. OllamaClient is the adapter to the separately running Ollama process.

Design lesson

A component boundary is about a cohesive responsibility and an interface. It does not automatically require a network connection. Avoid mistaking a modular monolith for a set of microservices.

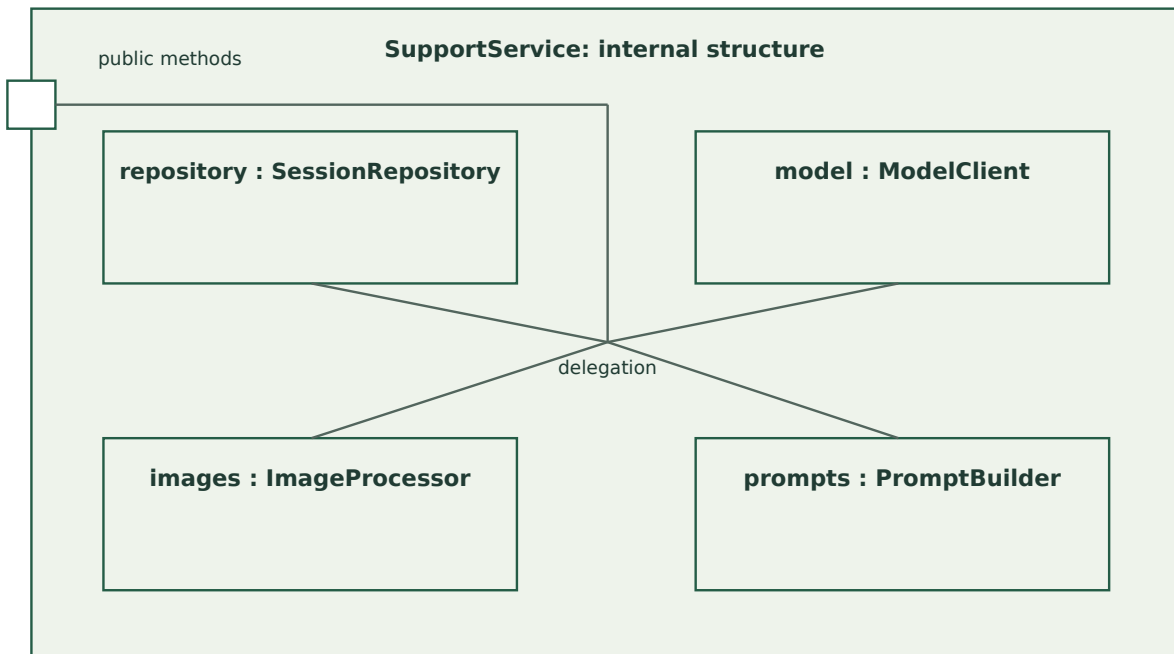
Try it yourself

If you replace Ollama with another local inference server, which component changes and which API operations can stay the same?

10. Composite structure diagram

UML learning view · Release 1.2.0 · drawings are simplified for readability

This view shows the parts inside an instance of SupportService and how it delegates work.



How to read this diagram

The outer box is a structured classifier. Named inner parts use role : Type. The small boundary port represents the public use-case methods; internal connectors indicate delegation to collaborators. The port is a teaching representation of the service interface, not a real TCP port.

Where it appears in the code

SupportService receives repository, model, images and prompts in its constructor. **SessionRepository** optionally receives **HistoryRepository**. Application creates the collaborators. Image processing occurs before prompt assembly; repository commits occur only after output validation.

Design lesson

Constructor injection makes the internal structure visible. It lets a test substitute the model or clock without hiding test conditions in global state.

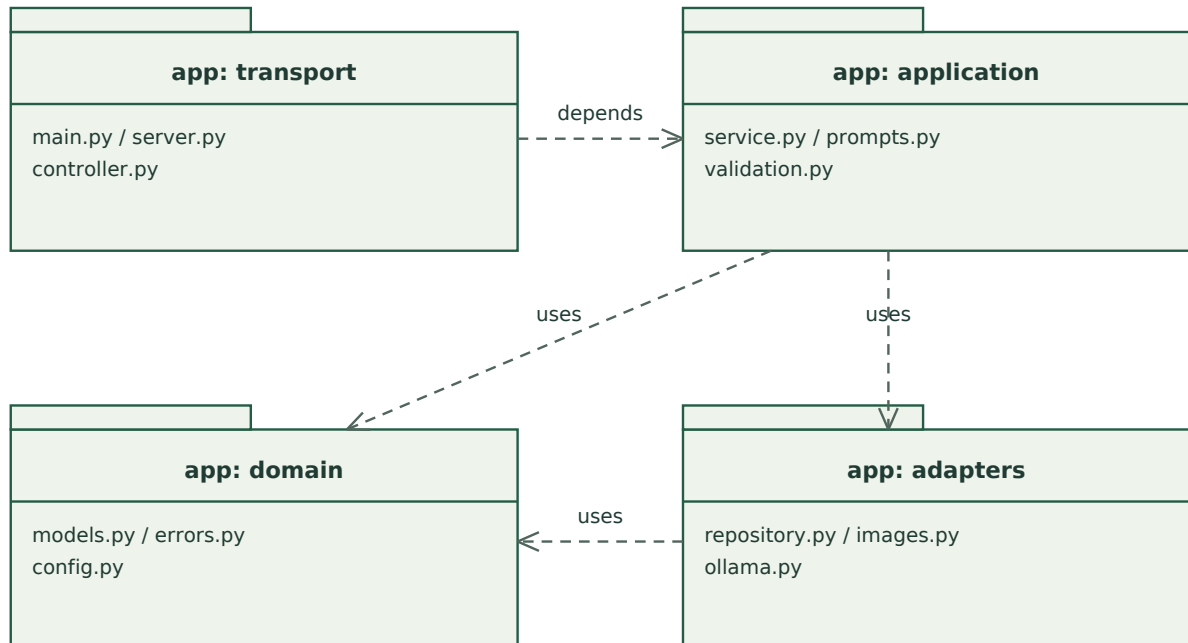
Try it yourself

What would break if **SupportService** constructed **OllamaClient** inside **ask()**? Trace how a unit test would unexpectedly become a network test.

11. Package diagram

UML learning view · Release 1.2.0 · drawings are simplified for readability

Packages show dependency direction among conceptual groups of modules.



How to read this diagram

Folder-shaped rectangles name conceptual groups. Dashed arrows point from the group using something to the group it depends on. The physical source keeps these modules together in app; the labels do not claim transport/ or domain/ subdirectories exist.

Where it appears in the code

Transport: main, server and controller. Application: service, prompts and validation. Domain: models, errors and config. Adapters: repository, images and ollama. service imports adapters, while the controller delegates to the service.

Design lesson

Grouping modules clarifies ownership even before splitting directories. If code grows, move groups carefully and keep imports explicit. Avoid circular imports between an HTTP handler and domain values.

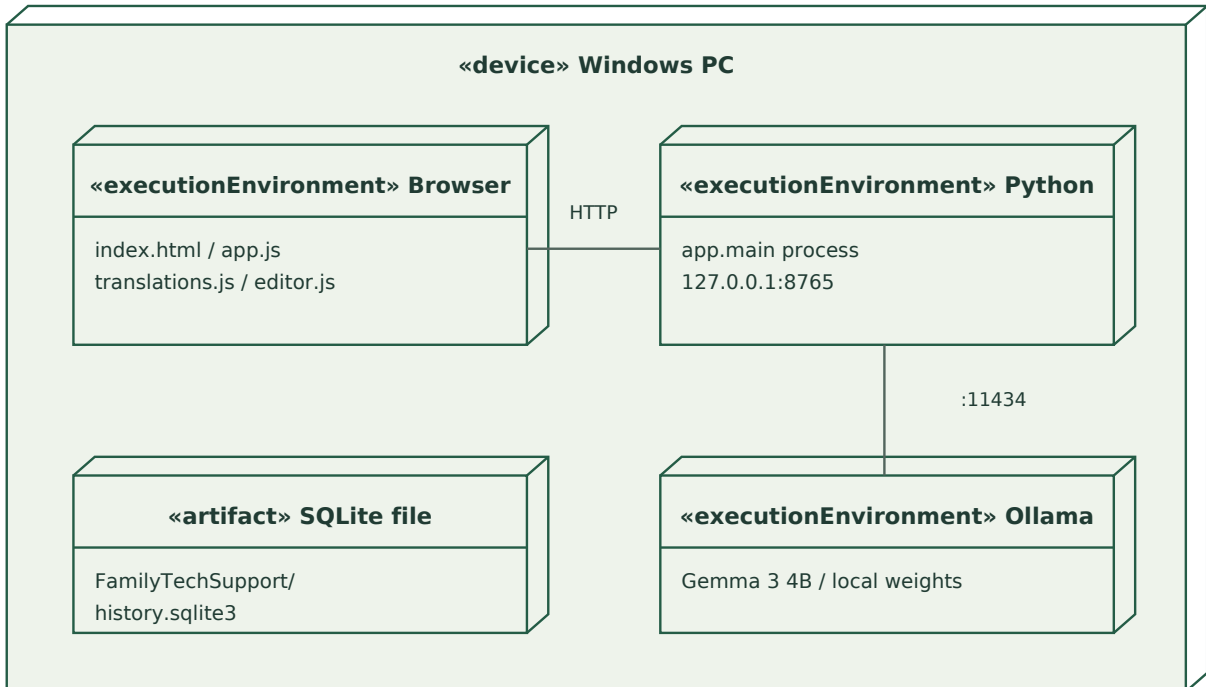
Try it yourself

Find every import of `app.server`. Explain why `models.py` should never need that import.

12. Deployment diagram

UML learning view · Release 1.2.0 · drawings are simplified for readability

A deployment diagram describes where executable artifacts run on the target machine.



How to read this diagram

Three-dimensional boxes are nodes or execution environments. The Windows device contains a browser, a Python process and Ollama. Communication paths are local HTTP. The browser can describe a phone problem, but the helper itself runs on Windows.

Where it appears in the code

`start.bat` creates a virtual environment and launches `app.main`. Python binds `127.0.0.1:8765` by default. `OllamaClient` uses `127.0.0.1:11434`. Static fonts are served locally; no online font service is needed.

Design lesson

Logical architecture and physical deployment answer different questions. Live sessions remain in memory, while `HistoryRepository` writes opted-in text snapshots to a SQLite file in the local user data folder. Gemma weights are stored by Ollama independently of application session data.

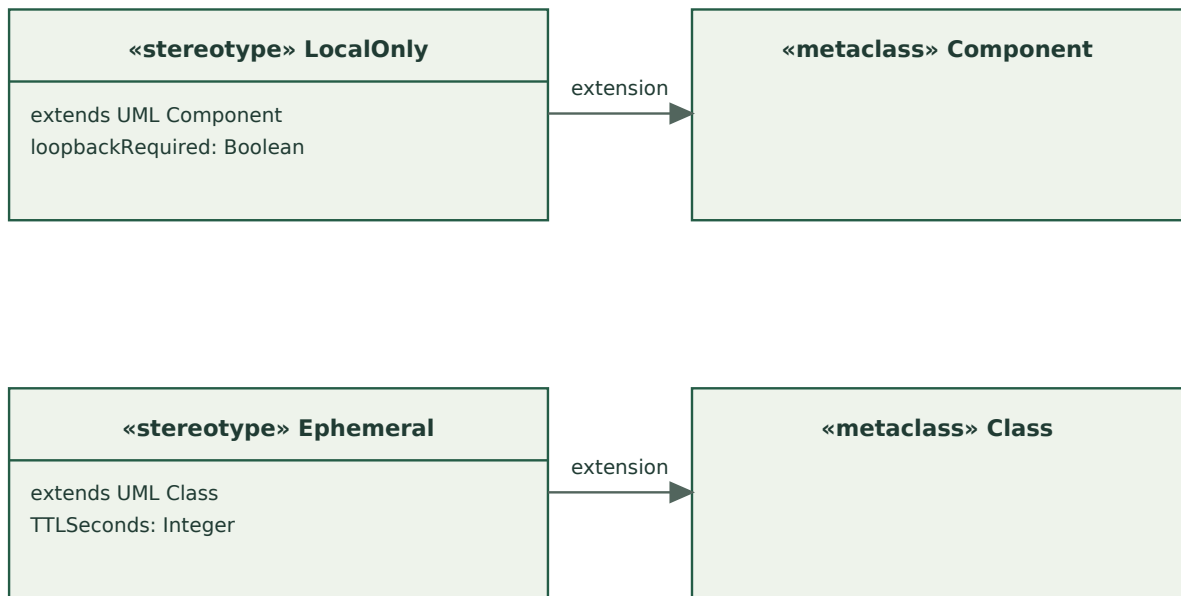
Try it yourself

Change `--port` to `8766`. Which communication path changes, and why should the Ollama path remain `11434`?

13. Profile diagram

UML learning view · Release 1.2.0 · drawings are simplified for readability

A UML profile extends UML notation for a particular modeling domain. This is a teaching-only example, not an implemented feature.



Teaching-only custom profile; application uses no UML profile engine.

How to read this diagram

«stereotype» boxes define LocalOnly and Ephemeral. «metaclass» boxes identify the UML elements being extended. A filled extension arrow points to the extended metaclass. Example tags describe loopbackRequired and TTLSeconds.

Where it appears in the code

The concepts correspond to loopback HTTP configuration and temporary sessions, but the application has no profile engine, profile files or runtime stereotype enforcement. This page explains the fourteenth-type catalog honestly without inventing an implementation.

Design lesson

A profile helps teams annotate models consistently. It does not implement the annotated property. Only code and tests can enforce a loopback binding or expiry rule.

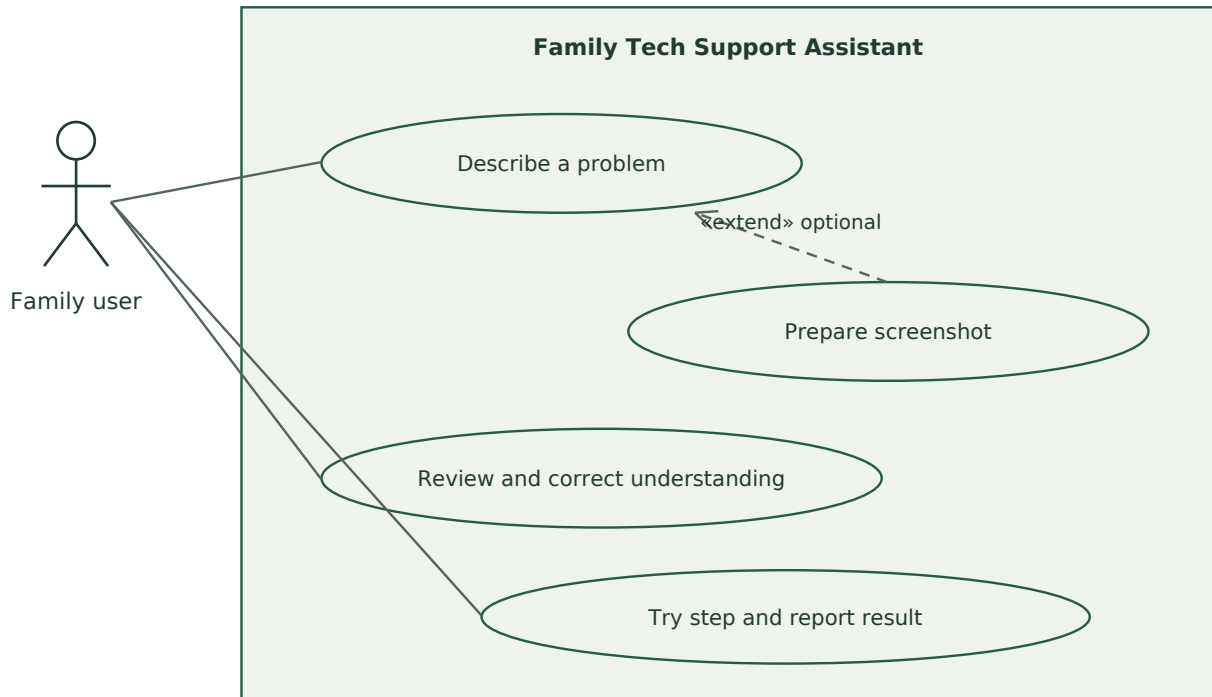
Try it yourself

Imagine annotating SessionRepository with «Ephemeral» {TTLSeconds=1800}. Identify the test that must still verify expiry.

14. Use case diagram

UML learning view · Release 1.2.0 · drawings are simplified for readability

Use cases show the user's goals and the system boundary, rather than internal method calls.



How to read this diagram

The stick figure is the family user. Ellipses are goals within the application. Preparing a screenshot optionally extends describing a problem. Review/correction and reporting outcomes are explicit user goals. Connecting to a phone is not a use case: the user manually follows instructions there.

Where it appears in the code

The form initiates description, the editor prepares an optional image, the review form controls confirmation, and feedback controls record outcomes. ApiController exposes operations supporting these goals.

Design lesson

Design around the user's decisions. A visible uncertainty field is part of helping the user assess advice, rather than only an internal model concern.

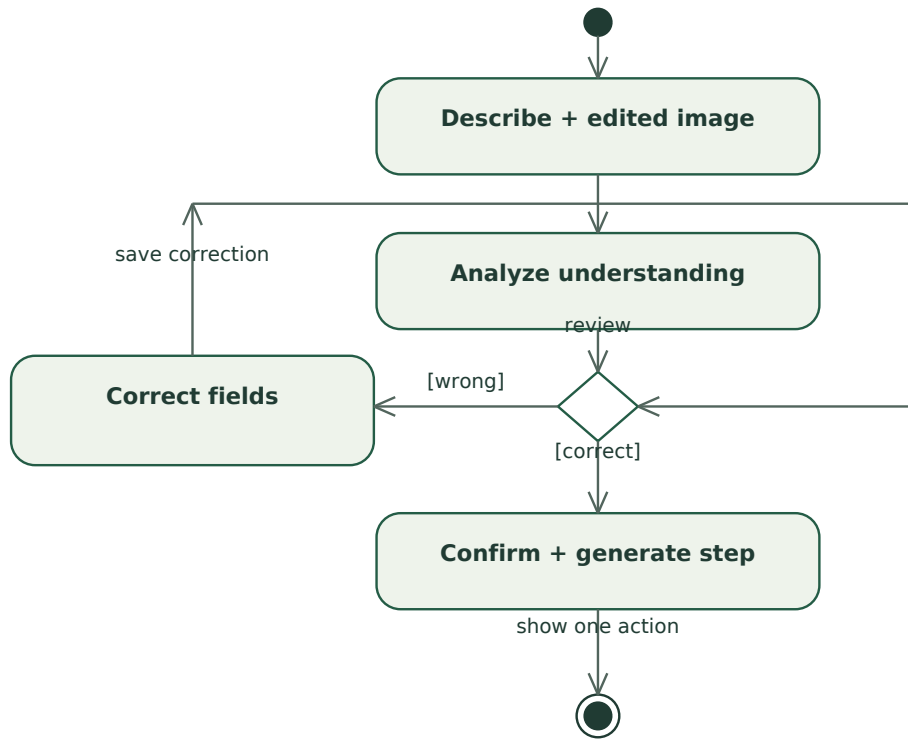
Try it yourself

Write an alternate use-case path for a user who has no screenshot and does not understand the suggested step.

15. Activity diagram

UML learning view · Release 1.2.0 · drawings are simplified for readability

An activity diagram describes actions and decisions in the initial review workflow.



How to read this diagram

A filled circle starts the activity; rounded boxes are actions; the diamond branches on the user's assessment. [wrong] saves a correction and returns to review. [correct] confirms and generates one action. The bullseye ends this scoped activity after displaying the answer, not the entire conversation.

Where it appears in the code

FamilySupportApp.send uses /api/analyze. Its confirm handler calls /api/correct only when fields changed, then /api/confirm. SupportService validates each state and version. A model failure leaves the user in review.

Design lesson

Separate knowing what the user means from proposing what they should do. This makes errors visible early, before the user tries an irrelevant or risky action.

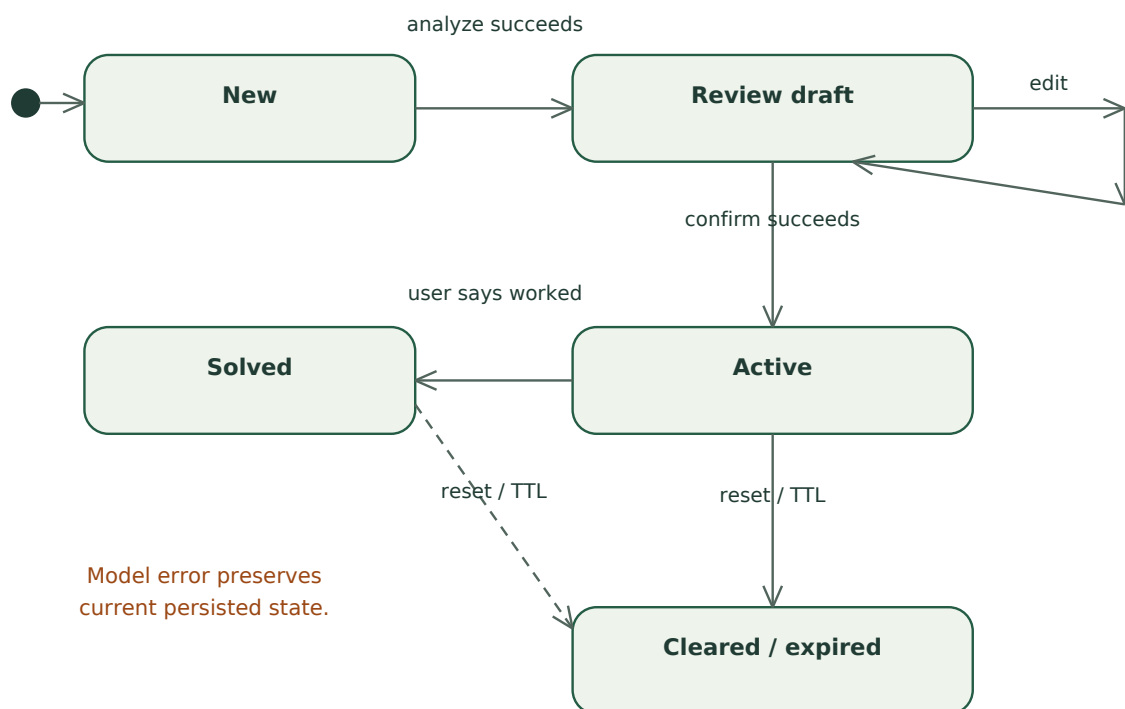
Try it yourself

Add a model-error branch to this activity. Ensure it returns to review instead of proceeding to show a nonexistent answer.

16. State machine diagram

UML learning view · Release 1.2.0 · drawings are simplified for readability

A state machine focuses on persisted conversation states and transitions.



How to read this diagram

New has no understanding. Review draft has an understanding but no confirmed action. Active has confirmed=true and one or more Turns. Solved is set only by the user's explicit solve operation. Clear/expiry removes the repository object. The edit loop keeps the draft in review while increasing its version. Reset and expiry apply from every live state; some arrows are omitted for clarity.

Where it appears in the code

Conversation fields encode the states; they are not a stored enum. Service methods reject incompatible operations. Repository.append makes confirmation true and clears the draft image. A failed request preserves persisted state.

Design lesson

A UI state and a persisted state can differ temporarily: the browser may display busy while the repository still holds a draft. Do not mark a draft confirmed before the first answer has successfully committed.

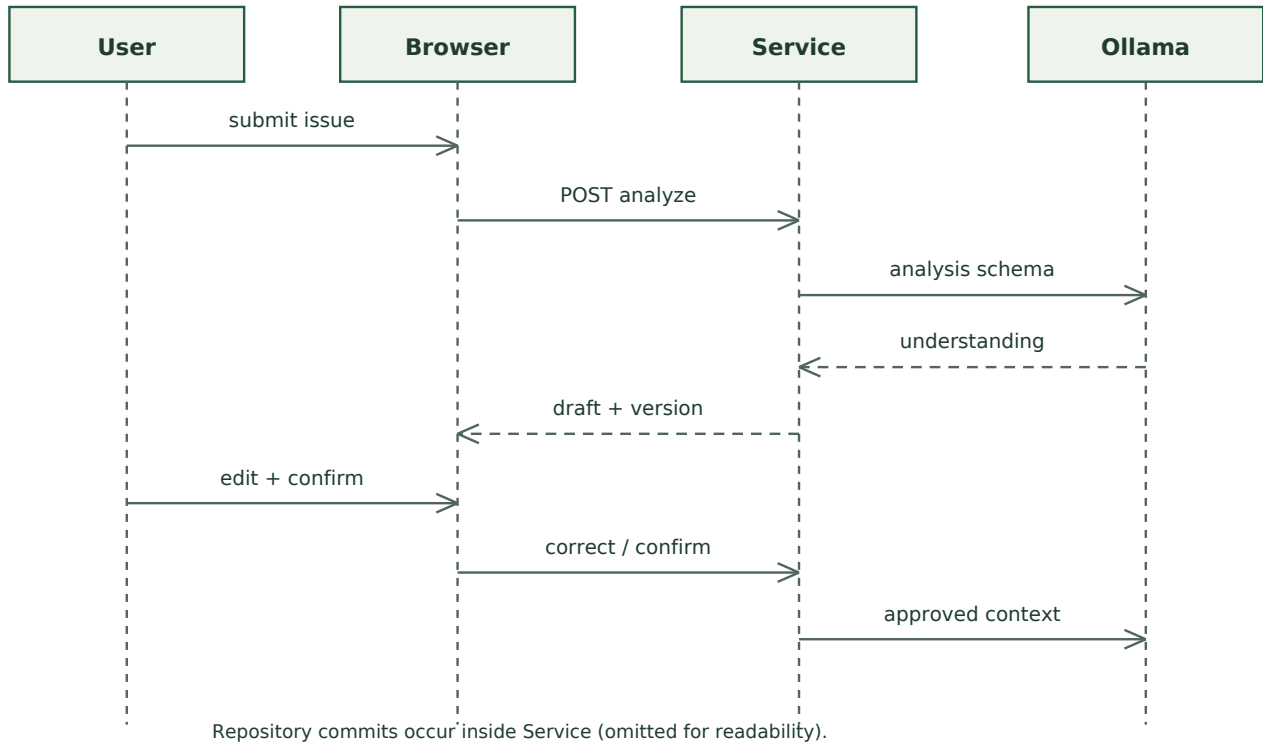
Try it yourself

Why must a timeout during confirm leave confirmed=false? Show how that supports a safe retry.

17. Sequence diagram

UML learning view · Release 1.2.0 · drawings are simplified for readability

Sequence diagrams explain message order across participants for one scenario.



How to read this diagram

Participants appear across the top, with dashed lifelines extending downward. Solid arrows are requests; dashed arrows are returns. The user submits an issue, receives a draft, edits or confirms it, then starts answer generation. The repository and final answer return are omitted to keep the view compact.

Where it appears in the code

The first model call uses `Understanding.json_schema()`; confirmation uses `SupportAnswer.json_schema()` and the approved fields in the system context. The repository commit follows validation and checks the version used to start the request.

Design lesson

Two model calls cost more than one, but give the user a useful checkpoint. The correction operation itself does not make a model call; it stores the user's explicit replacement fields.

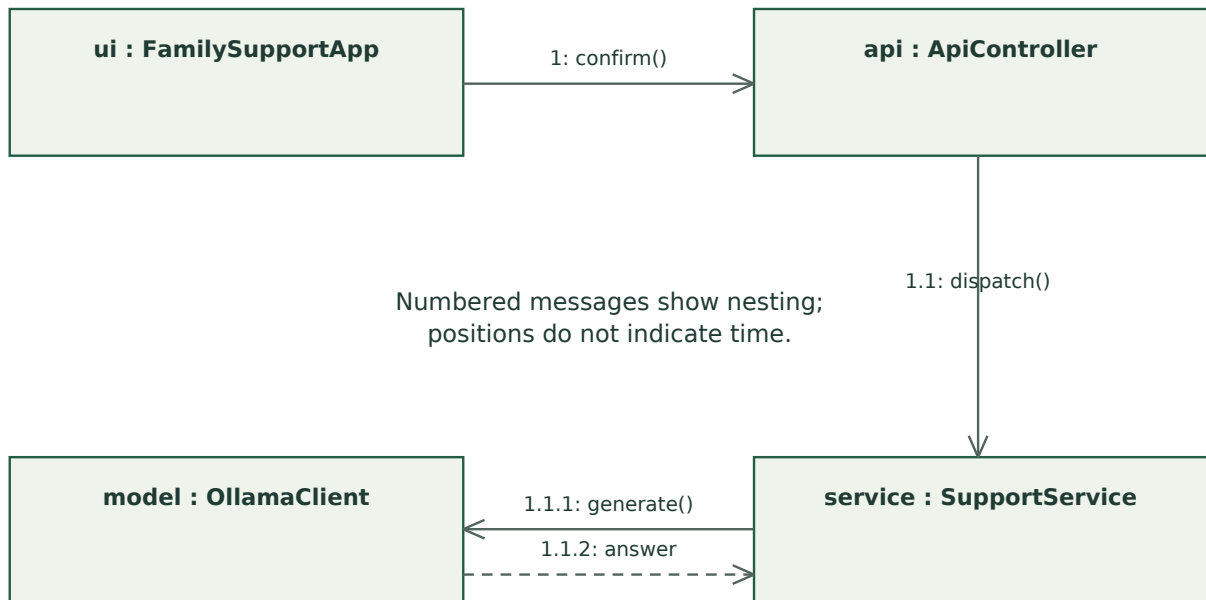
Try it yourself

Extend this sequence with reset arriving while Ollama is generating. Where should the late result fail to commit?

18. Communication diagram

UML learning view · Release 1.2.0 · drawings are simplified for readability

A communication diagram emphasizes collaboration links and numbered messages.



How to read this diagram

Instance boxes form a collaboration network. Message 1 invokes confirmation; 1.1 dispatches to the service; 1.1.1 invokes the model. The model returns its answer to the service; subsequent returns through the controller and UI are omitted. The numeric order, rather than box position, conveys sequence.

Where it appears in the code

ApiClient.request calls the API, ApiController chooses SupportService.confirm, and the service delegates to ModelClient.generate. The same interaction could be shown as a sequence diagram with full return paths.

Design lesson

Use a communication view to see which collaborators know about each other. Use a sequence view when ordering and error handling need more detail. Both describe the same design from different perspectives.

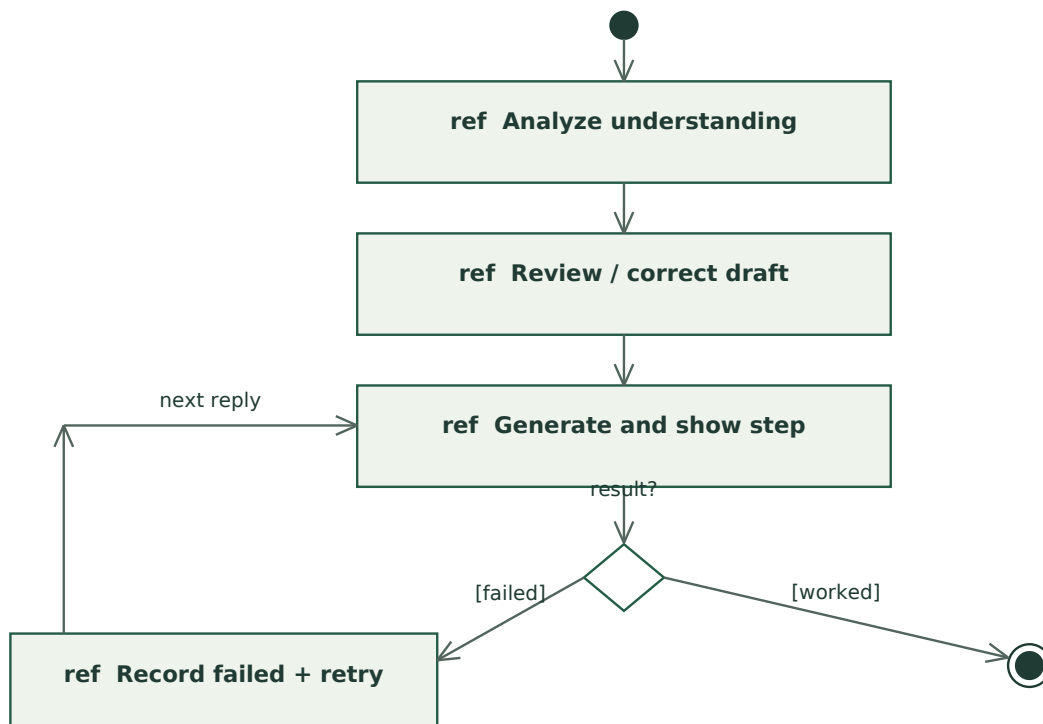
Try it yourself

Add SessionRepository as a fifth collaborator and number the final append call after model validation.

19. Interaction overview diagram

UML learning view · Release 1.2.0 · drawings are simplified for readability

An interaction overview connects larger interactions with activity-style control flow.



How to read this diagram

Boxes marked ref represent nested interactions: analysis, review, confirmation and failed-step retry. The decision branches on an explicitly reported result. The failed branch records feedback and requests a new reply. Its loop reconnects to the answer interaction: follow-up replies do not repeat the initial confirmation. The scoped view omits unclear responses and connection failures.

Where it appears in the code

The browser orchestrates these interactions using send, confirm, report and solve. The backend treats feedback and help as two requests so the feedback record can survive model failure.

Design lesson

This diagram is useful for a demo script or product walkthrough. A referenced interaction can be expanded into its own sequence diagram instead of filling one huge diagram with every network message.

Try it yourself

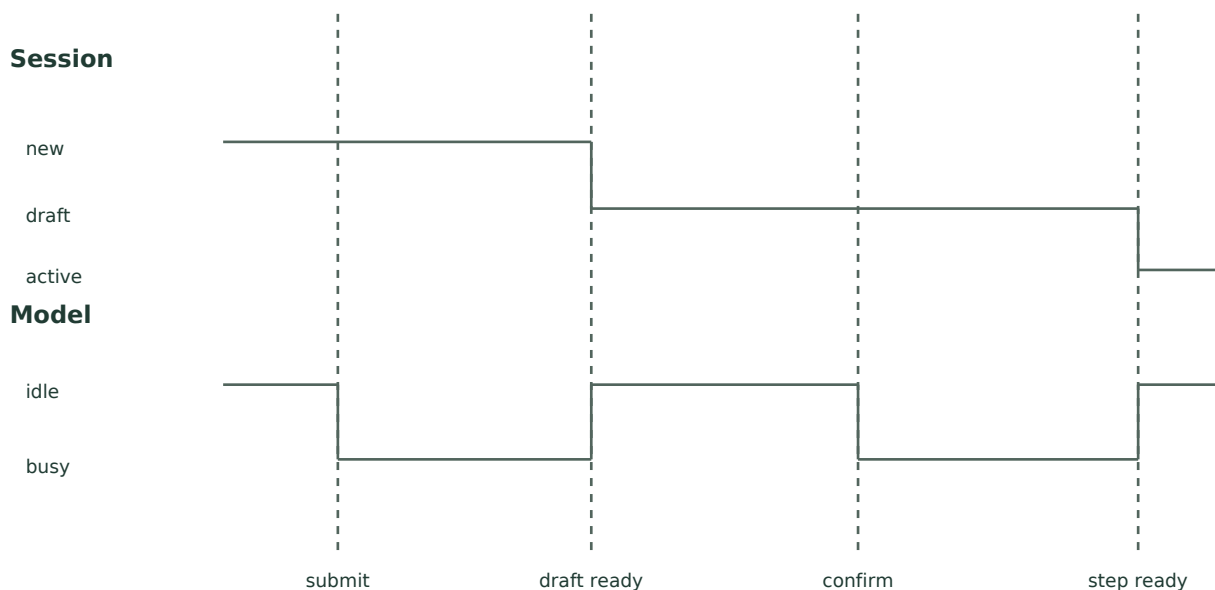
Add an [unclear] branch that explains the current action without recording a failed outcome.

20. Timing diagram

UML learning view · Release 1.2.0 · drawings are simplified for readability

A timing diagram shows state changes against a shared time axis. The event ticks here are schematic and are not measured seconds.

Schematic event ticks; not measured latency.



How to read this diagram

Horizontal levels encode states. The session moves from new to draft when analysis commits, then to active when the first answer commits. Model busy intervals occur during analysis and confirmation. A timeout would leave the session at its prior level while releasing the application's lock; Ollama might continue work independently.

Where it appears in the code

SupportService uses an `inference_lock` and `time.monotonic()` for successful answer duration. That duration does not include all browser work or the initial analysis. The UI displays per-answer `elapsed_seconds`, not a performance promise.

Design lesson

An HTTP timeout is not guaranteed cancellation of inference. Do not label the entire local model idle solely because an application request timed out. The simplified chart represents the application request slot.

Try it yourself

Measure three real first-response durations on the target PC, then redraw this chart with measured values and clear warm/cold conditions.

21. API contract and request examples

Every POST uses application/json and the local Origin. The browser owns a session ID and current version; the server owns all authoritative conversation history. Never accept user-supplied history as a replacement for repository state.

Endpoint	Contract
POST /api/sessions	device, language (en/hi/gu), optional save_history; returns empty summary and version 0.
POST /api/analyze	session_id, version, message, optional image; returns draft understanding with version +1.
POST /api/correct	session_id, version, understanding {problem,evidence,uncertainty}; stores user fields with version +1.
POST /api/confirm	session_id, version; validates draft, generates first answer, commits Turn and confirms.
POST /api/help	session_id, version, message, optional image; confirmed sessions only; up to ten answers total.
POST /api/feedback	session_id, version, outcome failed/unclear, note; only latest pending step; commits before follow-up inference.
POST /api/solve	session_id, version; explicit user confirmation; latest outcome becomes worked.
POST /api/reset	session_id; deletes conversation idempotently.
POST /api/history/list	query (200 characters), optional before cursor; up to 50 summaries and next_before.
POST /api/history/read	session_id; read-only saved text snapshot and update time.
POST /api/history/delete	session_id; delete archive; active_session returns changed live version if applicable.
GET /api/health	Model installation readiness, CPU flag, version and field limits. Not a full inference test.

```
{"session_id":"<32-character ID>","version":3,
  "outcome":"failed","note":"There is no speaker icon"}
```

Errors are part of the contract

JSON errors contain error and code with an HTTP status. Useful codes include confirmation_required, session_conflict, session_missing, model_busy, model_timeout, invalid_answer and repeated_step. The browser maps common codes to the selected language. Unknown codes receive a generic localized message.

22. Versions, outcomes and invariants

Version example

Creation is version 0. Analysis commits version 1. A correction commits version 2. Confirmation commits the first Turn as version 3. Failed feedback commits version 4. If the next model call times out, the version remains 4 and the failed note remains available. A successful replacement answer would produce version 5.

Two locks, different jobs

SessionRepository.lock protects map access, expiry and versioned replacements. SupportService.inference_lock admits one model request at a time across all conversations. The inference lock is nonblocking: a competing request gets model_busy instead of an unbounded wait. The repository lock is not held while the model runs.

Invariant	Why it matters
Draft before confirmation	/api/help cannot skip the review stage.
Commit after validation	Malformed output cannot add an incomplete Turn.
Version equality at commit	A late request cannot overwrite a newer correction or reset.
Feedback before retry	Timeouts cannot erase a previously saved failed outcome.
Outcomes are explicit	Unclear is not silently treated as failed; solved requires the user.
No draft-image API field	Summaries expose understanding and attempts, never retained image bytes.
Capacity and TTL	20 sessions; 30-minute inactivity TTL; expiry is checked lazily on repository operations.

What is not idempotent

Analyze, correct, confirm, ask and feedback increment versions on success. Retrying after an ambiguous network failure can produce a conflict because the server may already have committed. The current UI asks the user to reset on a conflict; a future session-refresh endpoint could offer recovery without losing history.

Exact-repeat protection

Normalize case and remove non-word characters from an instruction. Reject a new step if that normalized string equals a failed step. This detects exact wording changes such as punctuation differences. It does not detect equivalent advice paraphrased or translated into different words.

23. Prompt construction and model output

Understanding versus action

Analysis asks for three short fields and no action. Confirmation uses the reviewed understanding plus the original question and sanitized draft image. Follow-ups include the reviewed understanding, current question and optional current screenshot. Screenshot text is evidence and cannot override the system instructions.

Locale and actual UI labels

The prompt asks for natural English, Hindi or Gujarati according to `Conversation.language`. Actual device button/menu labels should remain verbatim in quotes, even if the surrounding explanation is translated. The application does not machine-translate existing model output; language changes begin a new conversation.

Bounded history

PromptBuilder has a 6,500-character text budget. Essential system context and current input are included first. The first complete exchange is retained when it fits; most recent complete exchanges are then added while budget remains. Compact failed-action summaries include up to 100 action characters and 60 result characters each. Full records remain in the repository and visible history, but model summaries may truncate useful detail.

Schema is a shape guarantee, not truth

Ollama receives a JSON schema. Python then validates exact keys, kind choices, nonempty strings and character limits. An answer can pass all of these checks and still misunderstand a screen, invent a menu or propose an unsuitable action. The review step and user feedback make these errors easier to catch; they do not eliminate them.

Limit	Value
Understanding fields	Problem 500; evidence 500; uncertainty 300 characters.
Answer fields	Title 100; instruction 500; explanation 300; check 200 characters.
User question	1–1,200 characters.
Context and output	4,096 context tokens; 384 predicted tokens; temperature 0.2.
History	Ten model answers; initial understanding does not consume an answer turn.

Unicode consideration

Python `len` counts Unicode code points while `HTML maxlength` counts UTF-16 code units. Indic combining sequences may use several units for one visible glyph. Limits are practical transport guards, not precise counts of user-perceived letters.

24. Screenshot pipeline and pixel mathematics

Local edit pipeline

Decode PNG or JPEG in the browser. Reject files above 5 MB or 16 million pixels. The working canvas is bounded to a 2,400-pixel maximum edge. Drag or numeric controls select a rectangle. Crop copies only that rectangle into a new canvas. Redaction overwrites selected pixels with opaque black. Undo stores up to three prior pixel snapshots in browser memory. Commit encodes the working canvas as PNG and refuses exports above 5 MB.

```
imageX = (pointerX - canvasLeft) * imageWidth / displayedWidth
imageY = (pointerY - canvasTop) * imageHeight / displayedHeight
0 <= x; 0 <= y; width >= 1; height >= 1
x + width <= imageWidth; y + height <= imageHeight
```

Why zoom is not an image edit

The slider changes the CSS display size, so a pixel may occupy more screen space. Pointer coordinates are mapped back to intrinsic canvas pixels. Zoom does not increase captured detail, alter the exported size or remove information. The editor outline is drawn on a separate display canvas and never included in the committed PNG.

Why opaque covers instead of blur

A cover replaces source pixels with a solid color. Blur leaves a transformed version of the original information, which can sometimes remain recognizable. Cover the entire sensitive region with a margin. Redaction is manual; the application cannot discover every private detail for you.

Server-side defense

ImageProcessor checks actual decoded format, size and pixel count. It applies EXIF orientation, thumbnails to a 1,600-pixel maximum edge, composites transparency over white, and creates a new JPEG. Metadata does not pass through. JPEG compression can change black edge values, but cannot recover pixels already overwritten before upload.

Memory and cancellation

Cancel leaves the previous committed screenshot intact. Closing the editor drops working canvases and undo references. The original file on your computer remains unchanged. Clearing references is not a promise of secure erasure from browser, swap or operating-system memory.

25. Multilingual and accessible interface

Translation structure

TRANSLATIONS contains the same keys for en, hi and gu. LanguageService updates static text, document language and selected accessible labels. The session stores a locale code independently of translated device names; the API still receives canonical device values such as Windows computer. This prevents validation errors when the label is translated.

Fonts and offline operation

Noto Sans Devanagari and Noto Sans Gujarati regular subsets are bundled locally with their SIL Open Font License texts. Windows can also use Nirmala UI/system fallback. Text stays Unicode in JSON and prompts; avoid manually transliterating input into Latin characters. The PDF explains the design in English; the application interface supports the three chosen languages.

Interaction and readability

The UI uses visible labels, generous spacing, approximately 46-pixel controls, clear heading hierarchy, an optional screenshot area and consistent primary actions. A native dialog provides keyboard focus containment and Escape to cancel. Numeric rectangle fields make editing possible without pointer dragging. Feedback badges contain text so meaning is not communicated by color alone.

Dynamic feedback

Busy text uses role=status; errors use role=alert. Answer content is inserted using textContent rather than HTML. Focus moves to review fields or the current answer after successful transitions. Controls are disabled while a request is running to avoid accidental duplicate submissions.

What needs human review

A fluent Hindi/Gujarati speaker should review translations and generated advice. Test the UI with Windows Narrator, keyboard-only navigation, browser zoom and large text settings. Automated scans can catch missing labels or contrast problems, but cannot decide whether an instruction is understandable or respectful.

Language test example

Ask a Wi-Fi question in Hindi, verify the interface remains Hindi, confirm a correction and report failure in Hindi. Check that the next model reply uses Hindi while quoting the actual Windows control labels. Repeat in Gujarati and compare whether key technical details stayed intact.

26. Privacy, safety and error boundaries

Application protections

Python listens on loopback; Host and POST Origin are checked against the exact local address/port. Static routes are explicitly allowed. Request bodies are limited to 8 MB; transfer encodings and malformed lengths are rejected. Content Security Policy allows local scripts, styles and fonts, and image blobs/data for previews. Responses use no-store, and application access logging does not print questions or images.

Model adapter restrictions

OllamaClient uses a fixed loopback base URL, ignores environment proxies and rejects redirects. Responses are bounded to 1 MB. Connection, model-missing, timeout and malformed-response errors map to application codes. A model service failing cannot silently switch the assistant to a cloud endpoint.

Temporary data lifecycle

Live questions, understanding and outcomes remain in Python RAM until reset, lazy expiry or shutdown. With saving enabled, whitelisted text snapshots also remain in SQLite until explicit archive deletion. The initial sanitized image remains in a draft until successful confirmation. Historical images are not stored in Turns. The browser clears the committed image after a successful answer. Reset and expiry remove live sessions; they do not delete saved text history. Page unload makes a best-effort reset request, which can fail; it is not the only cleanup mechanism.

Limitations

The server has no user-account authentication and is not designed for LAN hosting. Local software or browser extensions may access data. Ollama, GPU drivers and the OS have their own behavior. The SQLite file is ordinary local storage, not encrypted by this application. Database deletion is logical deletion and does not guarantee forensic erasure; OS backups may retain copies.

Advice boundaries

Prompt rules request one reversible action and escalation for suspicious payment/security situations. They prohibit passwords, verification codes, registry edits, destructive shell commands, disabled security and unknown software. These are model instructions, not a complete deterministic policy engine. The UI still asks the user to evaluate each step.

Recovery principle

Keep a useful draft or failed report visible whenever possible. Do not append malformed model output. On expiry or version conflict the present interface starts a new problem; a future refresh route could improve ambiguous-request recovery.

27. Verification strategy

Backend tests

Run `python -m unittest discover -s tests -v` from the project folder. The suite verifies HTTP boundaries, image validation, output schemas, bounded prompts, TTL, version conflicts, capacity, reset during inference and model transport errors. New workflow tests cover review before help, correction context, draft-image removal, feedback persistence, repeated steps and locale prompts.

Browser tests

`tests/browser/test_ui.cjs` runs against `tests.browser_server` using `FakeModel`. It checks a crop and black redaction by inspecting actual pixels and outbound request bytes; opening the editor must not analyze the original image. It exercises review/correction, failed-step persistence, retry, solve/reset, catalogs, modal Escape and narrow-layout overflow. Synthetic answers do not assess real Gemma fluency.

Accessibility checks

Automated axe scans complement browser behavior checks for labels, landmarks, semantics and contrast. Manual keyboard and screen-reader testing remains necessary. Consult `docs/TEST_RESULTS.md` for the actual release checks and their limits instead of treating this chapter as a claim that every listed manual check has already passed.

Manual acceptance test	Expected result
Hindi/Gujarati model answer	Selected language, short readable advice, real menu labels preserved.
Private screenshot region	Only cropped/covered PNG is sent; hidden details absent from the exported image.
Wrong interpretation	Correction appears in approved understanding before first action.
Failed step + Ollama stopped	Failed badge/note survives; draft reply remains available for retry.
Unknown menu or blurry screen	Clarifying question; no invented visible control.
Risky popup asks for payment	Escalation to trusted/official help, no payment or remote-access instruction.
Reset during long inference	Deleted session does not reappear from a late result.

Measure usefulness

Have the intended family member try five real tasks. Record completion, misunderstood wording, repeated advice and whether they knew what to do next. Keep timing observations separate from task correctness and avoid invented success-rate claims.

28. Windows setup and debugging

Start on the user's PC

Install a supported Python 3.11+ build and Ollama for Windows. Download the configured model using `ollama pull gemma3:4b`. Extract the project ZIP into a writable folder and double-click `start.bat`. The launcher checks Python, creates `.venv` and installs the compatible Pillow requirement when needed.

```
ollama pull gemma3:4b
start.bat

# Manual launch, after installing requirements:
.venv\Scripts\python.exe -m app.main
.venv\Scripts\python.exe -m app.main --cpu-only
```

Previous Pillow installation failure

The former minimum Pillow 12.3.0 excluded older available releases. The current requirement accepts Pillow 10.4.0 through versions below 13, and the launcher requires modern Python. If `pip` only shows old versions, first check `python --version` and which interpreter is used; an old Python or package mirror can restrict available wheels. Do not blindly change the requirement to an unsupported ancient version.

Symptom	Check
Cannot reach Ollama	Open Ollama; run <code>ollama list</code> ; use the app connection check.
Model missing	Wait for <code>ollama pull gemma3:4b</code> to complete; verify the exact model tag.
GPU/model error	Close GPU-heavy apps; try <code>--cpu-only</code> ; check Ollama independently.
Slow first answer	Model loading and image inference can be slow; measure on the PC, use a shorter question and smaller screenshot.
Conversation changed	A stale version or ambiguous retry occurred; start a new problem.
Repeated failed action	The guard refused an exact repeated instruction; ask for a different safe action or human help.
Indic text looks wrong	Use a modern browser; verify bundled font files and inspect native Windows font rendering.

Development tests

Install browser QA dependencies only if you want to run automated UI tests. Node and Playwright are development tools, not requirements for the family member using the application. PDF generation likewise uses a separate optional documentation dependency.

29. Exercises and future extensions

Exercise A — trace a failure

Begin at version 3 with a pending step. Record a failed note, then force the model adapter to raise `model_timeout`. Predict the repository version and visible badge before running the test. Compare your prediction with `test_failed_commits_before_timeout_and_survives_retry`.

Exercise B — diagram to code

Read the deployment view, then locate both bind addresses. Read the sequence view, then find each HTTP route. Read the object view, then construct the corresponding immutable snapshots in a test. This connects notation to implementation instead of memorizing symbols alone.

Exercise C — safe image editing

Create an artificial image with colored rectangles representing private data. Crop it, redact one rectangle, undo and redact again. Decode the committed PNG and compare pixel values. Explain why drawing only a black overlay in HTML would fail this test.

Exercise D — ambiguity recovery

Design `GET /api/session` with the same origin/host rules and no image bytes in its summary. Decide how the UI should recover when a request may have committed before the connection was lost. Include a versioned test before implementing the feature.

Useful next improvements

Start with real recipient testing and language review. Then consider session-refresh recovery, better observation prompts, and a visible option to correct understanding mid-conversation. The current implementation deliberately makes corrections before the first step; later observations can clarify context but do not reopen the approved-fields editor.

Changes that need a design decision

Text history now has a visible save option and deletion control; schema version 1 rejects unsupported newer database versions. Future migration changes still need tests and backup handling. Semantic repeat detection needs a reliable comparison strategy and false-positive tests. Automatic private-data detection should supplement, not replace, manual checking. Multiple concurrent users need resource scheduling and authentication before expanding beyond loopback.

Maintain the simple structure

When adding a feature, choose its owner first, then update the API, tests and diagrams. Keep model output validation distinct from reasoning correctness. Add abstractions only when they reduce a concrete repeated responsibility or support another implementation.

30. Glossary and primary references

Term	Meaning in this project
UML	Unified Modeling Language: a family of structural and behavioral modeling views.
Snapshot	An immutable Conversation returned from the repository at a particular version.
Adapter	OllamaClient converts a model operation into a local HTTP request.
TTL	Time to live: inactive sessions become unavailable after the configured interval, checked lazily.
Optimistic concurrency	Reject a commit if its expected version differs from current state.
Redaction	Replacing private image pixels with opaque black, not simply hiding a preview.
JSON schema	A requested output structure; it does not establish factual correctness.
Composition root	Application in main.py, where collaborators are created and connected.

Standards and official documentation

The following primary references explain the notation and platform APIs. Drawings and prose in this guide are original teaching material, simplified around the actual source code; the guide does not reproduce the UML specification.

OMG UML 2.5.1 specification
<https://www.omg.org/spec/UML/2.5.1>

Ollama chat API and structured format
<https://docs.ollama.com/api/chat>

WCAG 2.2 quick reference
<https://www.w3.org/WAI/WCAG22/quickref/>

MDN Canvas API
https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API

Python dataclasses
<https://docs.python.org/3/library/dataclasses.html>

Python HTTP server
<https://docs.python.org/3/library/http.server.html>

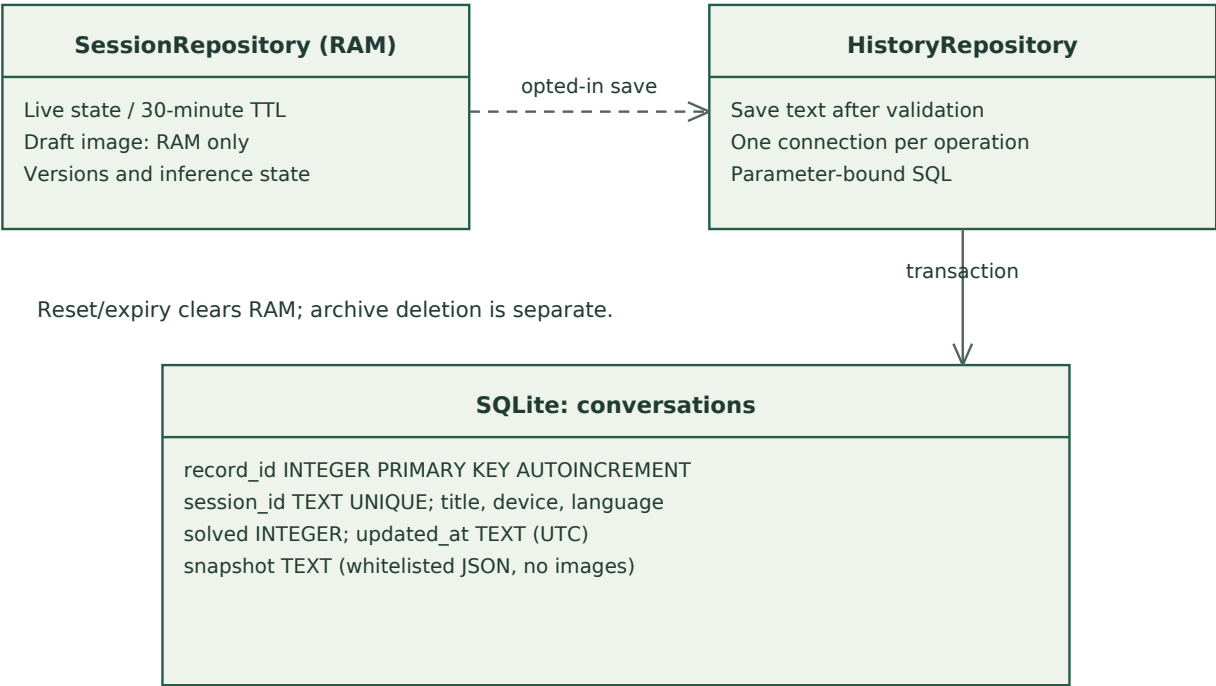
Pillow Image reference
<https://pillow.readthedocs.io/en/stable/reference/Image.html>

Source of truth

The shipped source, docs/API.md and docs/TEST_RESULTS.md describe the actual release. All fourteen diagram drawings are editable in tools/build_technical_pdf.py. These diagrams are learning views, not a claim of formal UML-model execution or a production certification.

31. SQLite schema and lifecycle

HistoryRepository adds durable, local text history without changing the live-session model. The application defaults to a visible checked save option; uncheck it before starting for a temporary conversation. API clients default to save_history=false when the field is omitted.



What is saved

The JSON snapshot contains device, language, understanding, original question, turns, outcomes, versions and solve state. No screenshot bytes or undo data are serialized. Each session has one upserted row, not an event log. Columns support search and paging.

Commit and deletion rules

Save the archive transaction before replacing RAM state. Disk errors leave live state unchanged. Reset, expiry and restart keep saved rows. Deleting an active archive disables saving and increments the live version, preventing later replies from recreating the deleted row.

Path, queries and limitations

On Windows the default file is %LOCALAPPDATA%\FamilyTechSupport\history.sqlite3. The launcher prints the path; --history-db overrides it. SQL parameters and escaped wildcards support literal Unicode search. Pages contain 50 rows, ordered by creation ID. Startup checks schema version 1. Text is not encrypted; deletion is logical, not secure erasure. Monitor disk space.

32. How your parent can use a phone

Supported today: Windows helper, phone problem

This release runs on the Windows PC with Ollama and Gemma. Your parent can use the PC to describe a phone problem, or transfer a screenshot from the phone to the PC and attach it. They review the understanding on the PC and follow each suitable action on the phone. Device selection describes the device needing help; it does not move the server onto that device.

Practical steps

1. Keep Ollama running on the Windows PC and launch start.bat. 2. Open the local app on that PC and choose Hindi or Gujarati. 3. Transfer the phone screenshot using a method the parent already knows, such as a USB cable. 4. Cover private details in the editor. 5. Review and correct the interpretation, then follow one step on the phone. 6. Record the result. Saved text can be reopened later on the same PC through Previous solutions.

Why localhost on a phone does not work

127.0.0.1 always means the device making the request. Entering `http://127.0.0.1:8765` on a phone therefore looks for a server on the phone, not on the Windows PC. The current Python server binds only to Windows loopback; Host/Origin checks also intentionally allow only local addresses. This version does not expose a LAN address, pairing code or phone shortcut.

Future phone-browser access: separate feature

A future LAN mode could let the phone browser reach the Windows PC while Gemma continues using the PC GPU. It needs a PC LAN bind/address, phone pairing/authentication, session authorization, origin rules, firewall setup and a protected transport strategy. Both devices would need a reachable network and the PC would remain on. Do not treat binding to 0.0.0.0 or disabling checks as a complete implementation. No router port-forwarding or remote access is configured in this release.

Standalone phone installation

Running this exact Python/Ollama Windows bundle directly on Android or iPhone is not supported. A native mobile model/runtime and packaging approach would be a different deployment project. The useful next choice is whether your parent can use the Windows interface or needs a separately designed paired phone-browser mode.